

CSC-Programming Languages -Assignment 1

Total: 100 points

Due date: Sept 26 at 11: 59.

Part1: (40 points)

All written answers must be typed. For parsing trees only, you may draw them by hand and insert a clear photo or scanned image into your document. Handwritten written response answers will receive a 20% deduction.

Q1. Prove that the following grammar is ambiguous (10 points)

$S \rightarrow A$

$A \rightarrow A + A \mid \text{id}$

$\text{id} \rightarrow a \mid b \mid c$

Q2. Using the following grammar: (14 points)

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

For following expression, show: $A = (A + B) * C$

1. a leftmost derivation.
2. a parse tree

Q3. Using the following grammar: (16 points)

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{number} \rangle$

$\langle \text{number} \rangle \rightarrow \langle \text{number} \rangle \langle \text{digit} \rangle \mid \langle \text{digit} \rangle$

$\langle \text{digit} \rangle \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Draw both a parse tree and an abstract parse tree for each arithmetic expression below.

1. $((2))$
2. $3 + 4 * 5 + 6 * 7$

Part 2: (60 points)

Description. In this assignment, implement a lexical analyzer in C that reads a source file and breaks the input into tokens. Your program should recognize identifiers, integer literals, assignment and arithmetic operators, parentheses, and semicolons. It should also detect invalid identifiers and unsupported symbols. In addition, the analyzer must ignore whitespace and skip both single-line (//) and block comments. Comments may appear between tokens and should not produce any output tokens.

Identifier rules. For this assignment, a valid identifier must begin with a letter or underscore (`_`). The remaining characters may contain letters, digits, or underscores. For example, `sum2`, `_count` are valid, while `2sum` should be reported as `INVALID_IDENTIFIER`.

Simple Example

Input	Expected Output
<code>sum = 10 + 25;</code>	IDENT sum ASSIGN_OP = INT_LIT 10 ADD_OP + INT_LIT 25 SEMICOLON ;

Submission.

Submit one ZIP folder containing: (1) your completed Part 1 Word document with answers to the questions, (2) your completed .c source file, (3) all required test input files, (4) a 3–5 minutes code walkthrough video, and (5) a README file. The README file should include the compiler used, the compiler version, the operating system or programming environment, and the steps or command used to compile and run the program. In the video, briefly explain the main functions, demonstrate at least one test case, and explain one important part of your implementation, such as comment handling or identifier validation. Make sure your program compiles and runs successfully in the compiler and environment described in your README file before submission.

Grading rubric for Part 2

Criterion	Points	What to Look For
Basic token recognition	10	Correctly recognizes identifiers, integer literals, <code>=</code> , <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , parentheses, and semicolons
Identifier validation	10	Correctly accepts valid identifiers and detects invalid identifiers such as <code>2sum</code>
Whitespace and comment handling	10	Correctly skips spaces, tabs, newlines, <code>//</code> comments, and block comments, including comments between tokens
<code>lex()</code> overall logic	10	Correctly builds lexemes, advances through the input, assigns token types, and reaches EOF properly
Invalid input handling	5	Correctly reports unsupported symbols and handles an unclosed block comment without crashing or looping forever

CSC-Programming Languages -Assignment 1

Required functions and program structure	5	Uses the required functions and follows the provided program structure appropriately
Testing and output	5	Program passes the provided test files and produces clear token/lexeme output
Code quality	5	Code is readable, reasonably organized, and uses meaningful comments/indentation
Total	60	